

Seção de Métricas na Landing do Leviticus

Leviticus · 18 de maio de 2026 · v1.0

1. Problema

A landing atual não apresenta nenhuma prova social. Um visitante que chega pela primeira vez não tem nenhuma evidência de que o produto está em uso real — só vê features e screenshots. Uma seção de métricas ao vivo resolve isso: mostra que outras igrejas já confiam na plataforma e que ela está ativa.

2. Objetivo

Adicionar uma seção na landing que exibe números reais do banco de produção, atualizados automaticamente, para transmitir confiança e encorajar novos cadastros.

3. Métricas disponíveis e recomendadas

Todas extraídas ao vivo do Supabase. Avaliação de impacto de cada candidato:

Métrica	Fonte no DB	Impacto para visitante	Recomenda do
Igrejas usando	<code>count(organizations)</code>	Alto — prova de adoção real	Sim
Músicos cadastrados	<code>count(auth.users)</code>	Alto — escala humana	Sim
Músicas na biblioteca	<code>count(songs)</code>	Médio — volume de conteúdo	Sim
Cultos criados	<code>count(playlists)</code>	Alto — uso real do produto	Sim
Ministérios ativos	<code>count(groups)</code>	Baixo — granular demais	Não
Músicas em cultos	<code>count(playlist_songs)</code>	Baixo — pouco intuitivo	Não

As 4 métricas recomendadas para v1:

- Igrejas usando o Leviticus
- Músicos na plataforma
- Músicas no acervo
- Cultos montados

4. Posicionamento na página

Entre e — logo após o visitante ver o produto em ação, antes de ler os recursos. Momento de maior receptividade para prova social.

5. Arquitetura técnica

Acesso aos dados

A landing usa a chave anon do Supabase, que não tem acesso a auth.users nem às tabelas protegidas por RLS. É necessário criar uma função SECURITY DEFINER no banco que retorna os agregados sem expor dados individuais:

get_platform_stats() — migration Supabase

```
CREATE OR REPLACE FUNCTION public.get_platform_stats() RETURNS json LANGUAGE
sql SECURITY DEFINER SET search_path = public AS $$ SELECT json_build_object(
'igrejas', (SELECT count(*) FROM organizations), 'musicos', (SELECT count(*)
FROM auth.users), 'musicas', (SELECT count(*) FROM songs), 'cultos', (SELECT
count(*) FROM playlists) ); $$; GRANT EXECUTE ON FUNCTION
public.get_platform_stats() TO anon;
```

Isso expõe apenas contagens agregadas — sem PII, sem dados sensíveis.

Fetch na landing

A landing é Next.js 15 com App Router. O page.tsx já é um Server Component async. A função é chamada server-side com cache de 1 hora:

```
const stats = await supabase.rpc('get_platform_stats', {}, { fetchOptions: {
next: { revalidate: 3600 } } })
```

Fallback: se a chamada falhar, a seção renderiza com null e omite os números — não exibe 0 nem quebra a página.

6. Formato de exibição

Seção em grid 2×2 (desktop) ou 2×2 empilhado (mobile). Cada célula exibe um número grande em destaque com label abaixo e ícone opcional. Estilo alinhado ao design atual da landing: fundo --card, borda --border, número em --text, label em --muted. Sem animações de contagem

(performance first).

Número	Label exibido
5	igrejas usando
5	músicos na plataforma
28	músicas no acervo
5	cultos montados

7. Contrato de props para o designer

O componente se chama Stats e recebe as seguintes props:

```
type StatsProps = { igrejas: number | null musicos: number | null musicas: number | null cultos: number | null }
```

Se null, a célula omite o número e pode exibir — ou ficar em branco. O designer tem liberdade total no visual — as props e o posicionamento (entre Showcase e Features) são os únicos contratos técnicos.

8. Entregas necessárias

#	Entrega	Responsável
1	Migration Supabase — get_platform_stats() RPC + GRANT para anon	Dev
2	apps/landing/components/Stats.tsx — componente recebe props, sem lógica de fetch	Design + Dev
3	apps/landing/app/page.tsx — chamada ao RPC e props passadas para	Dev
4	CSS — estilos da seção em globals.css	Design + Dev

9. Fora do escopo desta v1

- Animações de counter (número que 'sobe' ao entrar na viewport)
- Filtro por região
- Gráficos de crescimento
- Atualização em tempo real (WebSocket / Realtime)